

EMBEDDED SYSTEMS

# Smart Gate

## final implementation documentation

Agent-based vehicle access control -  
smart gate with license plate recognition

| Item          | Information                                                                   |
|---------------|-------------------------------------------------------------------------------|
| Team members  | Amir Džafić, Dino Tahirović, Imran Čustović                                   |
| Course        | Embedded Systems                                                              |
| Document type | Final implementation and description of changes compared to the specification |
| Date          | June 2026                                                                     |

## Summary

The system implements vehicle access control using an ESP32 controller, a Raspberry Pi 5 computer, MQTT communication, a camera for license plate OCR, Hermes/MCP functions, and Telegram notifications. In the final version, Tasmota firmware was not used. Instead, custom ESP32 firmware was developed to directly control the H-bridge, motor, ultrasonic sensors, and IR sensors.

## Contents

|      |                                                                   |    |
|------|-------------------------------------------------------------------|----|
|      | Summary                                                           | 1  |
| 1    | Project objective and final concept                               | 2  |
| 2    | Comparison of the original specification and final implementation | 2  |
| 3    | Final system architecture                                         | 3  |
| 4    | Hardware implementation and pinout                                | 4  |
| 4.1  | Motor logic                                                       | 4  |
| 4.2  | Sensor logic                                                      | 4  |
| 5    | Software components                                               | 5  |
| 6    | MQTT communication                                                | 6  |
| 6.1  | Example status message                                            | 6  |
| 7    | MCP functions and Hermes integration                              | 6  |
| 8    | Detailed system workflow                                          | 7  |
| 8.1  | Vehicle arrival                                                   | 7  |
| 8.2  | OCR and plate classification                                      | 7  |
| 8.3  | Allowed plate                                                     | 8  |
| 8.4  | Similar or unknown plate                                          | 8  |
| 8.5  | Opening                                                           | 8  |
| 8.6  | Holding the gate open                                             | 8  |
| 8.7  | Closing and safety                                                | 8  |
| 8.8  | Cycle completion                                                  | 8  |
| 9    | ESP32 controller states                                           | 9  |
| 9.1  | Timeouts and protections                                          | 9  |
| 10   | File structure and allowed_plates modules                         | 10 |
| 11   | Startup and testing                                               | 10 |
| 11.1 | System startup                                                    | 10 |
| 11.2 | Testing individual parts                                          | 10 |
| 11.3 | Examples of manual MQTT commands                                  | 10 |
| 12   | Limitations and conclusion                                        | 12 |

## 1 Project Objective and Final Concept

The objective of the project is to build a demonstration smart gate system that recognizes a vehicle at the entrance, reads the license plate, checks access rights, and then controls the opening and closing of the gate automatically or with user confirmation.

The final implementation is divided into two main parts:

- ESP32 performs low-level hardware control - sensors, motor, gate states, and MQTT messages.
- Raspberry Pi 5 performs the higher-level system functions - MQTT broker, camera-based plate reading, plate checking, Telegram notifications, and MCP functions through which the Hermes Agent controls the system.

This division is important because the ESP32 remains simple, fast, and responsible for safe gate movement, while the Raspberry Pi handles more demanding tasks such as OCR, communication with the user, and integration with the Hermes Agent.

## 2 Comparison of the Original Specification and Final Implementation

The original specification described the same basic scenario: a vehicle arrives at the entrance, the camera reads the plate, the agent checks the access rules, and the gate opens only if the vehicle is allowed. The final implementation keeps this idea, but changes the hardware implementation method and part of the control logic.

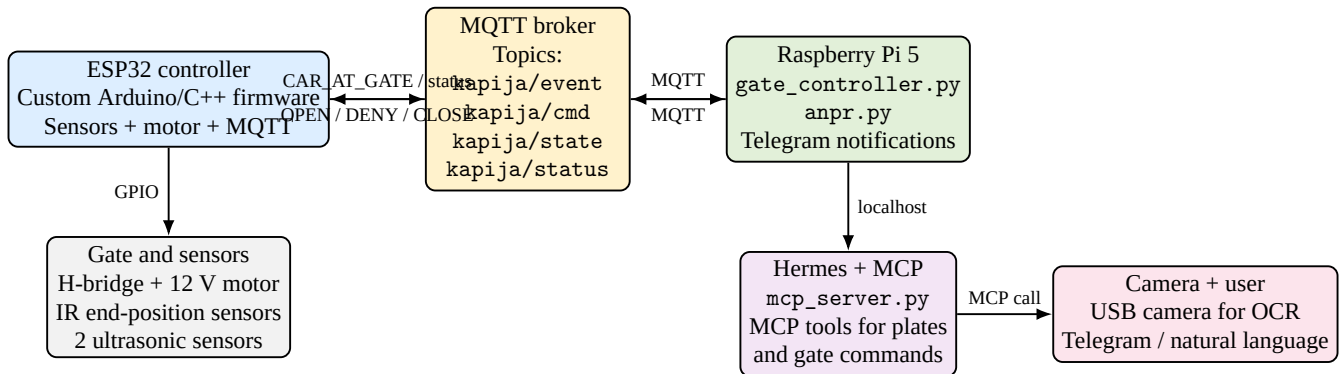
**Table 1:** Comparison of the original specification and final implementation

| Part           | Original specification                                       | Final implementation                                                         | Reason for change                                                                                 |
|----------------|--------------------------------------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Gate           | ESP32 + Tasmota + relays                                     | ESP32 with custom C++ firmware + CMOS/H-bridge                               | Custom firmware enables direct reading of sensors, gate states, and safety logic.                 |
| Motor          | Relay 1 for opening, relay 2 for closing                     | H-bridge controls the DC motor direction through two ESP32 outputs           | The H-bridge is more suitable for changing the direction of the motor.                            |
| Tasmota        | Planned as firmware for the ESP32                            | Not used in the final version                                                | It is not needed because the ESP32 code already handles WiFi, MQTT, GPIO, sensors, and the motor. |
| MQTT topics    | Tasmota cmd/stat/tele topics                                 | Project topics kapija/event, kapija/cmd, kapija/state, kapija/status         | The topics are adapted to the final system logic.                                                 |
| Camera         | Camera locally connected to Raspberry Pi                     | Same: USB camera on Raspberry Pi and Python OCR                              | This part remains conceptually the same.                                                          |
| Agent/MCP      | Hermes Agent calls MCP functions                             | The MCP server has tools for plates, pending decisions, and gate commands    | The final version adds practical tools for unknown plates.                                        |
| Plate database | Rules in conversation context, optionally persistent         | allowed_plates.json as a persistent local database                           | The list must survive program restart.                                                            |
| Unknown plate  | The gate remains closed and the user receives a notification | The user chooses through Telegram/Hermes: allow once, add and allow, or deny | An interactive decision was added for demonstration.                                              |

**Most important change:** the final system is not a Tasmota device. The ESP32 uses custom firmware. The Raspberry Pi is also not a Tasmota device, but a Linux computer that runs the MQTT broker, Python scripts, camera, and MCP server.

### 3 Final System Architecture

The final architecture uses MQTT as the link between the ESP32 and the Raspberry Pi system. The ESP32 sends events and status, while Raspberry Pi/Hermes sends commands. The camera is connected to the Raspberry Pi, not to the ESP32.



**Figure 1:** Final system architecture without Tasmota.

**Table 2:** System components and their role

| Component                        | Role                                                                                       |
|----------------------------------|--------------------------------------------------------------------------------------------|
| ESP32                            | Motor control, reading IR and ultrasonic sensors, MQTT communication, state machine.       |
| Raspberry Pi 5                   | MQTT broker, <code>gate_controller.py</code> , OCR through camera, Telegram notifications. |
| Hermes/MCP                       | MCP tools for adding/deleting plates and sending commands to the gate.                     |
| Camera                           | Vehicle image capture and OCR reading of the license plate.                                |
| Telegram                         | User decisions for unknown or similar plates.                                              |
| <code>allowed_plates.json</code> | The only real database of allowed plates.                                                  |

## 4 Hardware Implementation and Pinout

The ESP32 is connected to CMOS logic and an H-bridge that controls the DC motor of the gate. Two ultrasonic sensors detect a vehicle from the entry and exit sides. Two IR sensors are used to determine the gate position: closed, open, or between the end positions.

**Table 3:** ESP32 controller pinout

| Pin    | Name in code   | Purpose                                                   |
|--------|----------------|-----------------------------------------------------------|
| GPIO18 | ESP_OUT_A      | Output toward CMOS/H-bridge for one motor direction       |
| GPIO19 | ESP_OUT_B      | Output toward CMOS/H-bridge for the other motor direction |
| GPIO34 | IR_SENSOR_1    | End-position IR sensor                                    |
| GPIO35 | IR_SENSOR_2    | End-position IR sensor                                    |
| GPIO32 | ULTRA_IN_TRIG  | Trigger of the entry ultrasonic sensor                    |
| GPIO14 | ULTRA_IN_ECHO  | Echo of the entry ultrasonic sensor                       |
| GPIO33 | ULTRA_OUT_TRIG | Trigger of the exit ultrasonic sensor                     |
| GPIO26 | ULTRA_OUT_ECHO | Echo of the exit ultrasonic sensor                        |
| GPIO23 | LED_OPENING    | LED indication for opening                                |
| GPIO21 | LED_CLOSING    | LED indication for closing                                |
| GPIO22 | LED_STATUS     | Status LED when the motor is idle                         |

### 4.1 Motor Logic

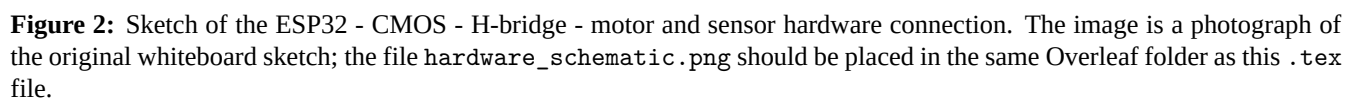
**Table 4:** Motor control logic through the H-bridge

| Action | ESP32 outputs                 | Effect                                        |
|--------|-------------------------------|-----------------------------------------------|
| STOP   | ESP_OUT_A=LOW, ESP_OUT_B=LOW  | Motor stopped                                 |
| OPEN   | ESP_OUT_A=LOW, ESP_OUT_B=HIGH | Motor turns the gate in the opening direction |
| CLOSE  | ESP_OUT_A=HIGH, ESP_OUT_B=LOW | Motor turns the gate in the closing direction |

### 4.2 Sensor Logic

**Table 5:** Interpretation logic for sensor signals

| Item             | Condition                     | Meaning                                             |
|------------------|-------------------------------|-----------------------------------------------------|
| IR sensors       | IR_ACTIVE_LOW=true            | The sensor is active when the digital input is LOW. |
| Gate closed      | ir1=true and ir2=true         | Both IR sensors are active.                         |
| Gate open        | ir1=false and ir2=false       | Both IR sensors are inactive.                       |
| Gate between     | ir1!=ir2                      | Only one IR sensor is active.                       |
| Vehicle detected | distance>0 and distance<=10cm | The ultrasonic sensor sees a vehicle.               |



## 6

**Table 6:** Software components of the system

| File                            | Role                                                                                                       |
|---------------------------------|------------------------------------------------------------------------------------------------------------|
| esp32_finalni.ino               | Firmware for ESP32. Connects to WiFi and MQTT, reads sensors, controls the motor, and manages gate states. |
| kapija/gate_controller.py       | Main Raspberry Pi program. Listens to kapija/event, starts OCR, classifies the plate, and sends OPEN/DENY. |
| kapija/anpr.py                  | Work with the USB camera and EasyOCR plate recognition.                                                    |
| kapija/allowed_plates.py        | Helper module for the main controller: normalization, checking, similarity, pending plate.                 |
| kapija/telegram_notify.py       | Sending Telegram messages to the user.                                                                     |
| kapija_mcp/mcp_server.py        | MCP server with tools called by the Hermes Agent.                                                          |
| kapija_mcp/allowed_plates.py    | Helper module for the MCP server: adding, deleting, and reading plates.                                    |
| kapija_data/allowed_plates.json | The only real database of allowed license plates.                                                          |
| kapija_data/pending_plate.json  | Temporary storage of the latest unknown or similar plate until the user decides.                           |

## 6 MQTT Communication

MQTT is the main communication layer between the ESP32 controller and the Raspberry Pi/Hermes part of the system. The ESP32 does not directly call OCR and does not directly communicate with Hermes; it sends events and waits for commands.

**Table 7:** MQTT topics and their purpose

| Topic         | Direction                    | Payload                                                          | Purpose                                                    |
|---------------|------------------------------|------------------------------------------------------------------|------------------------------------------------------------|
| kapija/event  | ESP32 -> Raspberry Pi        | CAR_AT_GATE                                                      | ESP32 reports that a vehicle has reached the entry sensor. |
| kapija/cmd    | Raspberry Pi/Hermes -> ESP32 | OPEN, DENY, CLOSE, STOP, RESET, STATUS                           | Commands for controlling the gate.                         |
| kapija/state  | ESP32 -> all                 | CLOSED, WAITING_DECISION, OPENING, OPEN, CLOSING, STOPPED, ERROR | Retained gate state.                                       |
| kapija/status | ESP32 -> all                 | JSON sensor status or text status                                | Telemetry and diagnostics.                                 |

### 6.1 Example Status Message

```
{
  "state": "OPEN",
  "ir1": false,
  "ir2": false,
  "gateOpen": true,
  "gateClosed": false,
  "gateBetween": false,
  "distanceIn": -1,
  "distanceOut": 8,
  "carIn": false,
  "carOut": true,
```

```

"anyCar": true,
"ocrRequested": false,
"blockUntilLeave": true,
"wifi": true,
"mqtt": true
}

```

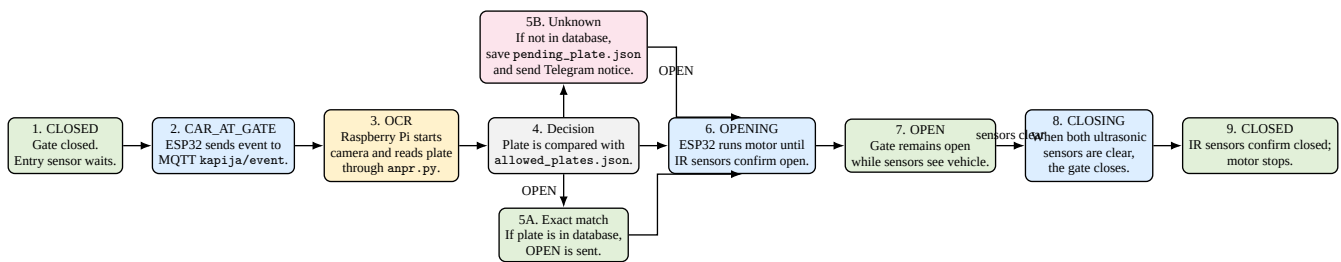
## 7 MCP Functions and Hermes Integration

The MCP server gives Hermes a set of tools. Hermes does not need to know GPIO pin details or directly control the ESP32. It calls an MCP function, and the MCP function sends the appropriate command over MQTT or changes the JSON plate database.

**Table 8:** MCP tools available to the Hermes Agent

| MCP function                                     | Description                                               |
|--------------------------------------------------|-----------------------------------------------------------|
| <code>lista_dozvoljenih_tablica()</code>         | Returns all plates that are currently allowed.            |
| <code>dodaj_dozvoljenu_tablicu(tablica)</code>   | Adds a plate to <code>allowed_plates.json</code> .        |
| <code>obrisi_dozvoljenu_tablicu(tablica)</code>  | Deletes a plate from <code>allowed_plates.json</code> .   |
| <code>provjeri_zadnju_nepoznatu_tablicu()</code> | Returns the contents of <code>pending_plate.json</code> . |
| <code>pusti_zadnju_nepoznatu_jednom()</code>     | Sends OPEN, but does not add the plate to the database.   |
| <code>dodaj_zadnju_nepoznatu_i_pusti()</code>    | Adds the pending plate to the database and sends OPEN.    |
| <code>odbij_zadnju_nepoznatu_tablicu()</code>    | Sends DENY and deletes the pending plate.                 |
| <code>otvori_kapiju()</code>                     | Manually sends OPEN to ESP32.                             |
| <code>zatvori_kapiju()</code>                    | Manually sends CLOSE to ESP32.                            |
| <code>zaustavi_kapiju()</code>                   | Manually sends STOP to ESP32.                             |

## 8 Detailed System Workflow



**Figure 3:** Workflow from vehicle arrival to gate closing. Unknown plate flow: the user chooses *allow once*, *add and allow*, or *deny* through Hermes/MCP functions.

### 8.1 Vehicle Arrival

When the gate is in the CLOSED state and the entry ultrasonic sensor detects a vehicle at a distance of up to 10 cm, the ESP32 sends the MQTT event `CAR_AT_GATE` to the topic `kapija/event`. After that, it enters `WAITING_DECISION` and does not start the motor until it receives the `OPEN` command.



## 8.2 OCR and Plate Classification

`gate_controller.py` receives `CAR_AT_GATE`, calls `procitaj_tablicu()` from `anpr.py`, receives the plate text and OCR confidence, normalizes the plate by removing spaces and special characters, and then compares it with `allowed_plates.json`.

### 8.3 Allowed Plate

If the status is `EXACT_MATCH`, Raspberry Pi sends the MQTT command `OPEN` to the topic `kapi.ja/cmd`. The ESP32 starts the motor in the opening direction.

### 8.4 Similar or Unknown Plate

If the status is `POSSIBLE_MATCH` or `NO_MATCH`, the system stores the data in `pending_plate.json` and sends a Telegram message. The user then chooses through Hermes whether to allow the vehicle once, add the plate and allow the vehicle, or deny access.

### 8.5 Opening

The ESP32 remains in the `OPENING` state until the IR sensors show that the gate is open. If this does not happen within the configured time, the system enters `ERROR_STATE`.

### 8.6 Holding the Gate Open

In the `OPEN` state, the motor is stopped. The gate remains open while at least one ultrasonic sensor sees a vehicle. This prevents closing while the vehicle is passing through.

### 8.7 Closing and Safety

When both ultrasonic sensors become clear, the ESP32 starts closing. If a vehicle is detected again during closing, closing is interrupted and the gate opens again.

### 8.8 Cycle Completion

When the IR sensors confirm the closed state, the motor stops, the state returns to `CLOSED`, and the system is ready for the next vehicle.

## 9 ESP32 Controller States

**Table 9:** ESP32 state machine states

| State            | Description                                                                                                       |
|------------------|-------------------------------------------------------------------------------------------------------------------|
| CLOSED           | Gate closed. Motor idle. The entry sensor can trigger CAR_AT_GATE.                                                |
| WAITING_DECISION | The vehicle is at the entrance. The ESP32 waits for the Raspberry/Hermes system decision. The motor does not run. |
| OPENING          | The motor opens the gate. Transition to OPEN occurs when the IR sensors indicate the open state.                  |
| OPEN             | The gate is open. Motor idle. The system waits until both ultrasonic sensors are clear.                           |
| CLOSING          | The motor closes the gate. If a vehicle appears, closing is interrupted and the gate opens again.                 |
| STOPPED          | Safety/neutral state when the position is not clearly open or closed.                                             |
| ERROR_STATE      | Error, most commonly opening or closing timeout. The motor is stopped.                                            |

### 9.1 Timeouts and Protections

**Table 10:** Time limits and safety resets

| Setting                       | Role                                                                                                      |
|-------------------------------|-----------------------------------------------------------------------------------------------------------|
| OCR_DECISION_TIMEOUT_MS=90000 | If no decision is received within 90 seconds, the gate remains closed and the waiting process resets.     |
| MAX_OPEN_TIME_MS=25000        | If the open state is not detected within 25 seconds, the system enters an error state.                    |
| MAX_CLOSE_TIME_MS=25000       | If the closed state is not detected within 25 seconds, the system enters an error state.                  |
| CAR_CLEAR_RESET_MS=1200       | A new OCR event is allowed only after the entry sensor has been clear for a sufficient period.            |
| WAITING_EMPTY_RESET_MS=8000   | If the vehicle leaves while the system is waiting for a decision, the system returns to the normal state. |

## 10 File Structure and allowed\_plates Modules

The system contains two files named `allowed_plates.py`, but this does not mean that there are two plate databases. The database is a single JSON file. The Python files are only helper modules used by two different programs.

```
/home/pi/hermes-timovi/tim09-root/
|-- kapija/
| |-- gate_controller.py
| |-- anpr.py
| |-- allowed_plates.py
| `-- telegram_notify.py
|-- kapija_mcp/
| |-- mcp_server.py
| `-- allowed_plates.py
`-- kapija_data/
    |-- allowed_plates.json
    `-- pending_plate.json
```

**Table 11:** Role of individual files

| File                                         | Why it exists                                                                                                                       |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>kapija/allowed_plates.py</code>        | Used by <code>gate_controller.py</code> . It has functions for plate checking, similarity comparison, and saving the pending plate. |
| <code>kapija_mcp/allowed_plates.py</code>    | Used by <code>mcp_server.py</code> . It has functions for adding, deleting, and listing plates through Hermes/MCP.                  |
| <code>kapija_data/allowed_plates.json</code> | The only real list of allowed plates. Both Python modules must work on this same file.                                              |

## 11 Startup and Testing

### 11.1 System Startup

1. Install the Python libraries required for MQTT, camera, OCR, and Telegram communication on the Raspberry Pi.
2. Start the MQTT broker, for example Mosquitto, so that the ESP32 can access the broker at the Raspberry Pi IP address.
3. In the ESP32 code, configure the WiFi SSID, WiFi password, and IP address of the Raspberry Pi MQTT broker.
4. Upload `esp32_finalni.ino` to the ESP32.
5. Start `gate_controller.py` on the Raspberry Pi. This program listens to `kapija/event` and sends commands to `kapija/cmd`.
6. Start `mcp_server.py` in the Hermes/MCP environment.
7. Configure the Telegram bot token and chat ID in `telegram_notify.py` or through the corresponding configuration.

### 11.2 Testing Individual Parts

### 11.3 Examples of Manual MQTT Commands

**Table 12:** Testing plan for individual system components

| Test                     | What is checked                                                                                                                         |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Motor                    | Manually send OPEN, CLOSE, and STOP through MQTT and verify the movement direction.                                                     |
| IR sensors               | Check <code>kapija/status</code> and confirm that both active means closed, and both inactive means open.                               |
| Ultrasonic sensors       | Place a vehicle in front of the entry and exit sensors and monitor <code>carIn</code> , <code>carOut</code> , and <code>anyCar</code> . |
| OCR                      | Run <code>anpr.py</code> directly and verify that the camera captures an image and returns plate text.                                  |
| Plate database           | Add a plate through an MCP function and verify that it appears in <code>allowed_plates.json</code> .                                    |
| Positive end-to-end case | Add a plate to the database, bring the vehicle to the entrance, and verify that the gate opens and closes.                              |
| Negative end-to-end case | Use an unknown plate and verify the Telegram notification and pending decision.                                                         |

```

mosquitto_sub -h <IP_RASPBERRY_PI> -t "kapija/#" -v
mosquitto_pub -h <IP_RASPBERRY_PI> -t "kapija/cmd" -m "OPEN"
mosquitto_pub -h <IP_RASPBERRY_PI> -t "kapija/cmd" -m "CLOSE"
mosquitto_pub -h <IP_RASPBERRY_PI> -t "kapija/cmd" -m "STOP"
mosquitto_pub -h <IP_RASPBERRY_PI> -t "kapija/cmd" -m "STATUS"

```

## 12 Limitations and Conclusion

**Table 13:** Known system limitations

| Limitation                          | Explanation                                                                                                                                       |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| OCR depends on lighting             | The camera must have stable light and the plate must be readable.                                                                                 |
| MQTT broker IP address is a setting | If the network or Raspberry Pi IP address changes, <code>RASPBERRY_MQTT_IP</code> must be changed in the ESP32 code.                              |
| Telegram token and WiFi data        | In a real submission and demonstration, they must be entered correctly, but they should not be published publicly.                                |
| Docker paths                        | The MCP container must have access to the same <code>kapija_data</code> folder as <code>gate_controller.py</code> .                               |
| Demo plate database                 | <code>allowed_plates.json</code> is a simple solution for demonstration; for production, a more secure database and authentication would be used. |

In conclusion, the final implementation fulfills the project objective: a vehicle is detected at the entrance, the camera reads the plate, the Raspberry Pi and Hermes/MCP part make a decision based on the plate database and user rules, and the ESP32 safely controls the gate motor and sensors. The biggest difference compared to the initial specification is the removal of Tasmota and the transition to custom ESP32 firmware, which enabled full control over the motor, sensors, safety states, and MQTT messages.